Coding_agent_harness 評估報告（二）：既有專案、模型組合與規劃的 self-refine

評估對象：Coding_agent_harness v4.1／v4.1.1，以及評測後的改版 v4.2｜模型（本機 Ollama）：gemma4:26b、Qwen2.5-Coder-32B-Instruct、DeepSeek-R1-Distill-Qwen-32B｜評估日期：2026 年 10 月 9～10 日｜詳細證據見《技術報告》｜第一次的評估見 `../benchmark_study/`

一、結論摘要

一句話：在大型真實專案裡照 issue 修正，harness 目前**不比**同一個模型直接寫好（約三成對兩成五）。換成「coding 很強」的 R1＋Qwen 組合**沒有比較好**，從零寫的任務還明顯變差。根據這次的發現做的 v4.2，讓 harness 在既有專案上快了一倍以上、改不進去的修改（SEARCH 對不上）少了四分之三，但解決的題數沒有變。下一步要修的是「驗證」本身：讓 harness 的「通過」真的代表修好了。

能力判定（更新第一次的評估）

任務類型	可以交給它嗎	證據（重點數字）	證據強度
小型、有測試的既有專案裡修改（第一次評估）	✅ 可以，要審查改動	14 個情境 37/37；同一個模型直接寫 71～82%	中
大型真實專案的 issue 修正（django、sympy、redis、fmt…）	⚠️ 只能當助手：找位置可以，改法要人確認	SWE-bench 24 題：harness 解決 7～8 題（29～33%），同一個模型直接寫 6 題（25%），差別不顯著。harness 改到正確檔案 18/24（直接寫 12/24），但 9 題「改對檔案卻沒修好」、4 題「修好了卻弄壞原本的測試」	中低（24 題、每個條件跑一次）
只給規格、從零寫整個專案（DevBench）	❌ 還不行（同第一次評估）	gemma4:26b 單元測試 14.7%（三次）、v4.2 23.3%（一次，沒有可信的差別）；R1＋Qwen 3.0%；Qwen 自己規劃 19.0%、v4.2 6.9%	中高

這次要回答的問題

1. **在既有專案上是不是比較可行？** 小而有測試的專案是；大型真實專案還不是。harness 的強項在大型專案上有出現：先探索、找到該改的檔案，而且從不交白卷。但它最大的優勢——「驗證、失敗就修」——在大型專案上失靈了：
   - 它選的測試指令常常不對（例如選了環境裡沒有的 pytest）。
   - 跑整套測試太久，或混進跟問題無關的失敗。
   - 自己寫的測試照自己的錯誤理解寫，所以錯的修改也會通過。

   結果是自己的驗證跟官方結果幾乎無關：自認通過的 4 題裡，3 題其實沒有解決。另外，大檔案的修改只看得到前 800 行，要改的地方在後面就改不進去。

   參考：同樣沒有微調的 Qwen2.5-Coder-32B，配公開的 agent 框架（OpenHands）在 SWE-bench Verified 上只有 7%，所以三成的量級本身不差（題目組成不同，只能看量級）。

2. **R1 規劃＋Qwen2.5-Coder worker 會不會比較好？** 不會。
   - 從零寫（DevBench）明顯變差：單元測試 3.0%，gemma4:26b 是 14.7%；12 個 repo 中 0 勝 7 負（p=0.016），每個 repo 的時間是 3.6 倍。
   - 既有專案（SWE-bench）差不多：8/24 對 7/24。唯一的亮點是 C++ 的 4 題全部解決（gemma 2/4）。
   - 主因是 harness 的寫檔協議照 gemma 的習慣調過。Qwen 建立檔案時就把功能寫完，後面的步驟變成原地打轉，佔 11～16% 的模型呼叫。
   - R1 當規劃者也沒有幫助：同樣的 Qwen worker，Qwen 自己規劃 19.0%、R1 規劃 3.0%（差別不顯著）。計畫的品質不是瓶頸。
3. **thinking 模式**：
   - deepseek-r1:32b 關不掉思考：送 think=false，它仍然先想。
   - gemma4:26b 可以開關；在簡單、要照格式回答的問題上，開了沒有變準，反而慢 3～30 倍。
   - qwen2.5-coder 沒有 thinking。

   建議主對話與 worker 關閉，規劃可以試開。
4. **規劃階段的 self-refine**：建議做「用程式檢查計畫、把具體的問題交回規劃者改一次」，成本低、也有文獻支持。不建議做「請模型自己再看一遍」：文獻顯示這種做法不可靠，而且 R1 的思考本身就在做這件事。依你的指示，等測試結束後再做，並當成獨立的改動量測。詳見第六節。
5. **評測後有沒有回頭優化程式？** 有，分兩部分：
   - **v4.1.1**：評測中找到 C++ 大綱掃描器遇到 gmock、jemalloc、doctest 這類大型標頭會卡住（正規表示式的災難性回溯）。已用一行修正，並加上回歸測試。DevBench 與第一次評估不受影響。
   - **v4.2**：根據既有專案的結果修改四件事——修改視窗以目標為中心、測試指令與範圍、改動要小、減少空轉。重測的結果（4.4 節）：
     - 修到的機制都有效：SEARCH 對不上少了 76%，每題時間的中位數 7.6 → 3.4 分鐘，卡住的 worker 少了將近一半。
     - 解決數沒有變（SWE-bench 7/24），DevBench 也沒有可信的變化。
     - 暴露出新的問題：有的驗證其實一個測試都沒跑到，卻判定通過。
     - Qwen 的空轉次數沒有減少，要改協議本身才行。
6. **Web Console**：主對話（非 coding）、Coding 規劃、Coding worker 三個角色可以分別設定模型與 thinking 開關。開關只在模型支援 thinking 時出現，設定會存檔。使用方式見附錄 A。
7. **待辦清單的 4 項**：已記錄在 `doc/待辦清單.md`，還沒有開始。

最重要的三個建議

1. 模型維持 **gemma4:26b**（規劃與 worker 都是）。
2. 在大型專案裡用 harness 時：
   - 在專案根目錄寫清楚怎麼跑測試（`AGENTS.md` 或 `.coding_agent.md`）。
   - 核准計畫前，檢查每一項的驗證指令。
   - 完成後一定看 diff。
3. 下一步依序做：

- 驗證要真的跑到測試（0 個測試不能算通過）。
- 既有專案的回歸檢查。
- 計畫的檢查器＋交回規劃者（含規格的介面清單）。
- 用 amr_base 的真實任務驗證。

二、評估目的與範圍

目的：

- 第一次評估用的 DevBench 量的是「只給規格、從零寫出整個專案」，連 GPT-4-Turbo 都只有個位數的通過率；日常的工作多半是在既有的專案裡修改。這次要回答：**在既有的真實專案裡，harness 行不行？**
- 換成一般認為 coding 很強的模型組合（DeepSeek-R1 規劃＋Qwen2.5-Coder worker），會不會比較好？
- 規劃階段要不要加上 self-refine？
- 同時完成 Web Console 每個角色的模型與 thinking 設定。

範圍：

- **既有專案**：SWE-bench（真實的 GitHub issue）24 題——Python 16 題、C 4 題、C++ 4 題。
- **從零寫**：DevBench 12 個 repo（同第一次評估）。
- **模型**：gemma4:26b、qwen2.5-coder:32b、deepseek-r1:32b，都是 Ollama 預設的 4-bit 量化。硬體是單一 RTX 5090，同時跑著 Isaac Sim。
- **不在範圍內**：雲端模型；修改 AI_agent_harness；使用者自己的 ROS 2 真實任務（待辦 4）。

三、評估方法（簡述）

評測	測什麼	規模	怎麼算分
SWE-bench 子集（新）	在大型真實專案裡照 issue 修正	24 題。Python：django、sympy、sphinx、astropy、xarray 等；C：redis、jq、micropython；C++：fmt、nlohmann/json	用官方的 Docker 環境與計分套件。修正問題的測試要通過，原本通過的測試不能被弄壞，兩者都達到才算「解決」
DevBench	只給規格、從零寫整個專案	12 個 repo（同第一次評估）	官方單元測試與驗收測試的通過率
thinking 量測	開／關 thinking 的效果	3 個模型 × 開關 × 5 題	有沒有思考、答對幾題、花多久

條件：

- harness＋gemma4:26b（第一次評估的建議設定）。
- harness＋R1 規劃＋Qwen worker（這次要測的組合）。
- harness＋Qwen 自己規劃（跟上一個條件只差規劃者，用來看 R1 的效果；只跑 DevBench）。

- 不用 harness：同一個模型直接寫。SWE-bench 用關鍵字檢索把相關的原始檔放進提示，一次寫出修改，這是 SWE-bench 論文的標準對照作法。

執行環境：每一題都在官方的 Docker 環境裡執行，沒有網路，harness 本身不用修改。它跑的測試、建置、編譯檢查都在題目的環境裡；agent 看不到評分用的測試與參考解答。計分環境先用參考解答驗證過：24 題全部解決。

比較方式：所有比較都用同一題（同一個 repo）配對，並附 95% 信賴區間。

四、主要發現

4.1 既有專案（SWE-bench 子集）

條件	解決	Python／C／C++	改到該改的檔案	什麼都沒改	每題時間
harness＋gemma4:26b	7/24（29%）	5/16、0/4、2/4	**18/24**	0	11 分鐘
harness＋R1 規劃＋Qwen worker	**8/24（33%）**	4/16、0/4、**4/4**	14/24	2	11 分鐘
直接寫：gemma4:26b	6/24（25%）	4/16、0/4、2/4	12/24	6	0.2 分鐘
直接寫：Qwen2.5-Coder	6/24（25%）	4/16、1/4、1/4	14/24	4	0.6 分鐘

- **差別都不顯著**。同一題配對比較，gemma 的 harness 對直接寫：「只有 harness 解決」5 題、「只有直接寫解決」4 題（p=1.0）。兩邊解決的題目重疊很少，表示單次的結果受隨機性影響很大；24 題分辨不出 5～10 個百分點的差距。
- **harness 找得到地方**：改到正確檔案 18/24（直接寫 12/24），而且從不交白卷。
- **但改對沒有比較多**：gemma 的 harness 有 9 題「改對檔案但沒修好」、4 題「修好了但弄壞原本的測試」。它改得比較多：原始碼改的行數中位數 18 行，參考解答 7 行；另外 24 題中有 17 題加了測試檔。
  - 例：django-12276 把方法移到正確的類別（跟參考解答一樣），卻順手改了基底類別的參數與回傳邏輯，結果沒修好、還弄壞一個原本的測試。
- **自己的驗證失靈**：harness 自認通過的 4 題裡 3 題其實沒解決；自認沒過的 19 題裡，5 題其實解決了。原因有三：
  - 選錯測試指令：sympy 兩題都選了環境裡沒有的 pytest。
  - 跑整套測試：太慢，或混進無關的失敗。
  - 自己寫的測試照自己的錯誤理解寫。
- **依難度**：修正大約 15 分鐘內的簡單 issue，harness 4/6（直接寫 2/6、1/6）；15 分鐘～1 小時的四個條件都只有 1～3/8；1 小時以上的全部 0/2。
- **大型檔案改不進去**：檔案超過 800 行、要改的地方又在後面時，產生修改的模型看不到原文。gemma 的「SEARCH 對不上」125 次中有 44 次是這個原因，R1＋Qwen 是 103 次中 41 次（v4.2 已修）。
  - 例：xarray-3677 的計畫完全正確，修正那一項卻一直改不進去，最後只交出測試。

4.2 換成 R1 規劃＋Qwen2.5-Coder worker

條件	DevBench 單元測試	DevBench 驗收測試	DevBench 每個 repo 的時間	SWE-bench 解決
harness＋gemma4:26b（DevBench 3 次）	14.7%	**23.5%**	**3.6 分鐘**	7/24
harness＋R1 規劃＋Qwen worker（DevBench 2 次）	3.0%	11.8%	12.8 分鐘	**8/24**
harness＋Qwen 自己規劃（DevBench 1 次）	**19.0%**	11.8%	6.4 分鐘	—
直接寫：gemma4:26b	4.3%	17.6%	0.2 分鐘	6/24
直接寫：Qwen2.5-Coder	3.4%	13.7%	1.0 分鐘	6/24

DevBench 明顯變差。R1＋Qwen 對 gemma4:26b，12 個 repo 中 0 勝 7 負 5 平（p=0.016）——這是這次研究唯一顯著的模型差別。

原因是 harness 的寫檔協議跟 Qwen 的習慣不合：

- worker 分兩步：先決定要做什麼（這一步只看得到摘要、看不到檔案內容），再由另一次呼叫寫出檔案。
- Qwen 建立檔案時就把整個功能寫完。下一步又被要求「實作某函式」時，它把一樣的內容送回來，工具回報「沒有修改」。
- Qwen 當 worker 的三次執行，各有 52～63 次這種空轉，佔 11～16% 的呼叫，幾乎全部發生在同一個 job 稍早建立的檔案上（gemma 是 3～5%）。
- Qwen 的工具錯誤率（31%）、驗證失敗、卡住的次數也都比 gemma 多。

SWE-bench 差不多，C++ 特別好。修 issue 多半是改既有檔案的幾行，空轉的機會少，Qwen 的程式能力就顯現出來（C++ 4/4）。但它改錯地方的題目較多（7 題），自己的驗證一樣失靈。

R1 當規劃者沒有幫助：

- 同樣的 Qwen worker，R1 規劃 3.0%、Qwen 自己規劃 19.0%（0 勝 3 負 9 平，p=0.25）。差距幾乎全部來自 graph-cpp：文件自己寫了兩種名稱，選對就通過、選錯就全掛。
- R1 的計畫格式良好、很少需要修補，但計畫品質本來就不是瓶頸。
- R1 的技術報告也自承它在結構化輸出（JSON）、工具使用、軟體工程任務上沒有優勢；我們用的 32B 蒸餾版沒有任何軟體工程的評測。

速度：

- GPU 同時跑著 Isaac Sim，32B 模型要把 KV cache 量化成 8-bit 才放得進去，約 42 tokens/秒。
- gemma4:26b 是 MoE（每個 token 只用約 4B 參數），快好幾倍。
- R1 與 Qwen 交替時還要換模型。

4.3 thinking 模式

模型	支援 thinking	送 think=false 時	簡單問答（5 題）	建議
deepseek-r1:32b	是（一定會想）	**仍然會思考**（關不掉）	開 3/5、關 2/5；要求 JSON 格式時會出現離題的答案	開關沒有作用；只適合當「慢、但計畫格式穩定」的規劃模型
gemma4:26b	是	不思考（0.2 秒）	開 3/5（慢 3～30 倍）、關 5/5	主對話、worker 關；規劃可以試開
qwen2.5-coder:32b	否	—	關 4/5（送 think=true 會被拒絕）	設定畫面不顯示開關

5 題太少，只能說「在簡單、要照格式回答的問題上，沒有看到 thinking 的幫助」。在困難的規劃上有沒有幫助，這次沒有證據；會思考的 R1 規劃也沒有比較好。

4.4 評測中找到的 harness 問題與改版

問題	影響	處理
C++ 大綱掃描器遇到大型標頭（gmock、jemalloc、doctest、expat）會卡住：正規表示式的災難性回溯	SWE-bench 7 題受影響：探索專案卡 10 分鐘，規劃時可以卡到整個 job 逾時；修正前跑完的 4 題沒有一題解決	**v4.1.1 已修正**（一行）並加上回歸測試。重跑這 7 題解決 2 題，平均 8 分鐘。DevBench 與第一次評估的專案沒有這種檔案，結果不受影響
大型檔案的修改視窗只有前 800 行	gemma 的「SEARCH 對不上」125 次中 44 次目標在第 800 行之後（R1＋Qwen：103 次中 41 次）；有的題目計畫正確卻改不進去	v4.2：以 worker 剛看過的範圍、對不上時最像的段落、或提到的函式為中心
驗證用的測試指令與範圍不對	harness 自己的驗證跟官方結果幾乎無關	v4.2：認得 django、sympy 自己的測試程式；不能照抄佔位符；沒有 pytest 算環境問題；大型專案的整體驗證改成依序跑每一項自己的驗證指令
改得太多	原始碼改的行數是參考解答的 2.5 倍；4 題弄壞原本的測試	v4.2：規則改成「只改必要的地方」；重現問題的測試跟修正放在同一項
Qwen 的空轉	11～16% 的模型呼叫浪費掉	v4.2：寫入後告訴 worker 檔案裡現在有哪些類別與函式；空轉時明說「已經有了，可以回報完成」

v4.2 的量測：SWE-bench（gemma4:26b，24 題）

指標	v4.1／v4.1.1	v4.2
解決	7/24	7/24（新解決 4 題、反而沒解 4 題）
每題時間（中位數）	7.6 分鐘	**3.4 分鐘**
每題模型呼叫	86	61
SEARCH 對不上（目標在第 800 行之後）	125（44）	**30（7）**
每題卡住被結束的 worker	1.30	0.71
原始碼改的行數（中位數；參考解答 7）	18	12
自己驗證「通過」的題目，官方也解決	1/4	6/16

指標	v4.1／v4.1.1	v4.2
自己驗證「沒過」的題目，官方確實沒解決	14/19	7/8

- **修到的機制都有效**：
  - 4.1 節舉的例子 xarray-3677 這次 1.2 分鐘就解決。
  - SEARCH 對不上少了 76%，卡住的 worker 少了將近一半，每題的時間少了一半以上。
- **解決數沒有變**：新解決 4 題、反而沒解 4 題，兩次都解決的只有 3 題。單次執行的隨機性大於改版的效果；這一點跟 harness 對直接寫的比較一樣。
- **自己的驗證，「沒過」變得可信，「通過」仍然不可信**。v4.2 會跑專案自己的測試，所以通過的題目從 4 題變成 16 題，但其中只有 6 題官方真的解決：
  - 大多數跑的是 agent 自己新寫的測試，照自己的理解寫。
  - 3 題其實一個測試都沒跑到（django 的測試標籤寫錯，回報「Ran 0 tests … OK」），其中一題帶著會讓 django 當掉的修改「通過」。

  這是下一步要修的：驗證沒有跑到任何測試時，不能算通過（路線圖第 2、5 項）。

v4.2 的量測：DevBench（從零寫）

條件	單元測試	驗收測試	對之前（同一個 repo 配對）
gemma4:26b	14.7%（三次）→ 23.3%	23.5% → 33.3%	2 勝 2 負 8 平（不顯著）；差別幾乎全來自 graph-cpp 這次選對了文件裡兩種名稱中的一個
Qwen 自己規劃	19.0% → 6.9%	11.8% → 11.8%	1 勝 2 負 9 平（不顯著）；差別來自 graph-cpp 這次選錯

- v4.2 的修正都針對既有專案，DevBench 的檔案都很短，本來就不預期有大的變化。
- **Qwen 的空轉沒有減少**（60 → 57 次）。v4.2 的提示讓 Qwen 遇到空轉後直接回報完成的比例從 15% 提高到 39%，重複同樣動作而被判卡住的比例從 10% 降到 4%。但空轉本身的起因沒有被提示解決：決策的步驟看不到檔案內容，而 Qwen 建立檔案時就已經把功能寫完了。要解決，需要改協議本身（路線圖第 4 項）。

小結：v4.2 修到的機制都看得到作用，但一次的量測分辨不出分數的變化。SWE-bench 上 v4.1 與 v4.2 各解決 7 題、只有 3 題相同；DevBench 一個 repo 的名稱選對或選錯，就差 12～14% 的測試。改版的效果要用重複的量測確認（第九節）。

五、Gemini 建議的逐項檢查

完整的對照表（含 harness 的現況）見《技術報告》6.3 節。

Gemini 的說法或建議	判斷	依據
26B 直接寫比透過 harness 拆解更穩定	✗ 與數據不符	DevBench：harness 14.7%、直接寫 4.3%（7 勝 0 負）；SWE-bench 兩者差不多（29% 對 25%）。harness 的呼叫次數確實多很多（每個任務約 60～90 次，直接寫 1～5 次）
Gemma 是通用模型，會格式漂移、頻繁重試	✗ 不成立	回覆格式由 Ollama 的結構化輸出保證，三種模型的空白回覆都是 0；gemma 的工具錯誤率反而比 Qwen 低

Gemini 的說法或建議	判斷	依據
R1 當規劃者最好	✗ 沒有證據	同樣的 worker，R1 規劃沒有比 Qwen 自己規劃好；R1 的報告自承結構化輸出、工具使用、軟體工程沒有優勢
Qwen2.5-Coder 當 worker 最好	△ 部分成立	改既有程式的能力確實好（SWE-bench C++ 4/4），但跟 harness 的寫檔協議不合（空轉 11～16%），在這台機器上也慢 1.8～3.6 倍
動態路由：簡單任務跳過規劃	△ 收益小	規劃只佔 job 時間的一小部分；跳過規劃就沒有「核准計畫」讓人審查的時機
規劃前先唯讀探索	√ 已經有	harness 改到正確檔案 18/24，直接寫 12～14/24——這是大型專案上最明顯的優勢
思考與動作分離	√ 已經有	思考放在獨立欄位；worker 只拿到計畫的項目。這次另外加了每個角色的 thinking 開關
用真實環境的回饋（測試、linter、diff）	√ 已經是核心設計	要補的是「測試指令與範圍選對」（v4.2 修了一部分）
失敗時交回規劃者重新規劃	○ 可以做	屬於「計畫的檢查器＋交回規劃者」的延伸（第六節）
邊做邊規劃，不要一次拆完	○ 設計取捨	跟「人核准計畫」相衝突；可以改做「某一項失敗時重新規劃剩下的項目」
換成 R1＋Qwen-Coder-32B	✗ 不建議（在目前的 harness 上）	DevBench 明顯變差（p=0.016）、慢 3.6 倍；SWE-bench 差不多

六、規劃階段的 self-refine：評估與建議

你的想法：worker 可以用測試精煉自己的結果；規劃階段能不能也「寫完計畫 → 檢查 → 修改」？

現況：規劃流程已經有一半。

- 程式會檢查計畫：路徑存不存在、驗證指令能不能跑、任務列出的檔案有沒有項目負責……
- 有問題時由程式自己修補，再交給你核准。
- 缺的是「把問題交回規劃者，讓它自己改一次」。

文獻與這次的數據：

- **文獻**：
  - 沒有外部回饋的自我修正大多沒有效果，有時反而變差（Huang 等人 2024；Kamoi 等人 2024 的綜述）。
  - 程式的自我修復，受限於模型給自己的回饋品質（Olausson 等人 2024）。
  - 規劃要靠外部的驗證器檢查，再把錯誤回饋給模型（LLM-Modulo，Kambhampati 等人 2024）。
- **這次的數據**：
  - 會思考、思考時本身就在自我檢查的 R1，當規劃者沒有比較好。
  - 計畫需要程式修補的情況不多：DevBench gemma 6/36、R1 2/24、Qwen 1/12；SWE-bench 2/23、3/24。

- 主要的失分都不是「規劃者多想一次」看得出來的：從零寫是介面名稱跟規格不一致，既有專案是改不對、驗證訊號錯誤。
- 規劃只佔 job 時間的一小部分，多一次規劃呼叫的成本不高。

建議

做法	建議	理由
請規劃者再看一遍自己的計畫（純自我批評）	✗ 不做	文獻顯示不可靠；R1 的思考已經在做；失分的主因不是多想一次看得出來的
用程式檢查計畫、把具體的問題交回規劃者改一次（LLM-Modulo 式）	√ 建議做	成本低：沒有問題就不多花呼叫，最多交回 1 次；而且跟待辦 1、2 是同一件事

要檢查並交回規劃者的內容：

1. 現有的計畫檢查：路徑、驗證指令、檔案分配。
2. **介面覆蓋**（待辦 1）：規格裡的類別、函式、常數都要分配到某一項，名稱拼法要跟規格一致。
3. **驗證範圍**：既有專案的驗證指令要用專案自己的測試程式，而且只跑相關的測試。
4. 既有專案：計畫提到要改的函式、類別，要真的存在於指定的檔案。
5. **驗證指令要真的跑得到測試**：規劃時先試跑一次，跑到 0 個測試或跑不起來的，就交回規劃者。v4.2 有 3 題的驗證一個測試都沒跑到，卻判定通過；這種錯誤「請模型再想一次」抓不到，用程式一跑就知道。

時機：依你的指示，等這次的測試結束後再做，並當成獨立的改動來量：同一個模型，DevBench 至少跑 3 次，加上 SWE-bench 子集，比較改動前後。

七、使用建議與風險

7.1 模型與 thinking 的設定

在 Web Console 標題列的「🧠 模型」設定。

角色	建議模型	thinking	理由
主對話（非 coding）	gemma4:e4b（快），或 gemma4:26b（跟 coding 共用、不必換模型）	關	回覆要照格式、要快
Coding 規劃	gemma4:26b	關（可以試開）	R1 規劃沒有比較好，而且慢；規劃只佔 job 時間的一小部分，試開的成本不高
Coding worker	gemma4:26b	關	Qwen 跟寫檔協議不合；worker 的呼叫次數多，開 thinking 會慢很多

要在 Isaac Sim 執行時用 32B 模型，Ollama 要開 KV cache 的 8-bit 量化才放得進 GPU，設定方法見附錄 A。

7.2 在既有專案上使用

1. 在專案根目錄放一份開發說明，寫清楚怎麼建置、怎麼跑測試、怎麼只跑一部分測試（例如 `python tests/runtests.py <模組>`）。harness 會讀 `AGENTS.md`、`CLAUDE.md`、`CODING.md` 或 `.coding_agent.md`。
2. 核准計畫前檢查（畫面上可以改）：
   - 每一項要改的檔案。
   - 驗證指令是不是只跑相關的測試。
   - 整體驗證是不是用專案自己的測試程式。
3. 完成後一定看 diff，特別是：
   - 「順手」改到的函式簽名、基底類別、原本的測試。
   - 超過 800 行的檔案的修改（v4.2 之前）。
4. 不要只看 harness 自己的「驗證通過」。在大型專案裡，它自己的驗證跟真實結果的一致性很低，要用你信任的測試再跑一次。
5. 比較實際的用法：先讓 harness 找位置、提出修改，人確認改法之後再驗證。

7.3 風險

風險	說明	降低的方法
自己的驗證不可靠（大型專案）	v4.1 自認通過的 4 題裡 3 題其實沒有解決；v4.2 通過的 16 題裡 10 題沒有解決，其中 3 題一個測試都沒跑到	指定測試指令與範圍；看驗證的輸出有沒有真的跑到測試；人自己跑一次信任的測試
改太多、弄壞原本的行為	v4.1 改的行數是參考解答的 2.5 倍，24 題中 4 題弄壞原本的測試（v4.2：1.7 倍、3 題）	看 diff；v4.2 加了「只改必要的地方」的規則
留下除錯用的檔案	agent 會在專案根目錄留下自己的除錯或重現腳本（v4.2：23 題中 3 題）	看 diff、刪掉不需要的新檔案
換模型不一定變好	協議與提示是照 gemma 調的；Qwen 有 11～16% 的呼叫在空轉	換模型前先用評測（`evals/`）量過，不要只看排行榜
agent 執行的程式可以連網路	同第一次評估（待辦 3）；這次的既有專案評測在沒有網路的容器裡跑，可以當成實作的參考	待辦 3
結果的不確定性	24 題、每個條件只跑一次；單次的隨機性很大	重要的決定以重複的量測為準（第九節）
資料污染	SWE-bench 的題目與修正都公開在 GitHub 上，可能在模型的訓練資料裡	用自己的任務驗證（待辦 4）

八、改進路線圖（更新）

每一項都要用同一套評測量測改動前後：固定版本，每個條件至少跑 2～3 次，並附信賴區間。

#	項目	狀態	為什麼（證據）	量測什麼
✔	C++ 大綱掃描器的災難性回溯	已修正（v4.1.1）	7 題卡住（4.4 節）	回歸測試

#	項目	狀態	為什麼（證據）	量測什麼
1	修改視窗以目標位置為中心	v4.2 已做，有效：SEARCH 對不上 125→30 次（第 800 行之後 44→7）	SEARCH 對不上 125 次，其中 44 次的目標在第 800 行之後	SEARCH 對不上的次數、卡住的 worker、解決數
2	既有專案的測試指令與範圍	v4.2 做了一部分：改跑專案自己的測試，「沒過」變得可信（8 題中 7 題確實沒解決）。還沒做：沒有跑到任何測試時不能算通過；django 的測試標籤要對照 `tests/` 目錄；規劃時先試跑一次驗證指令，看跑不跑得起來、要多久	自己的驗證跟官方結果幾乎無關	自己的驗證跟官方結果的一致性
3	改動要小	v4.2 已改規則：原始碼改的行數中位數 18→12，弄壞原本測試 4→3 題	原始碼改的行數中位數 18，參考解答 7	patch 大小、弄壞原本測試的題數
4	減少空轉（讓協議適用於不同模型）	v4.2 只改了提示：Qwen 的空轉次數沒有減少（60→57），但遇到空轉後比較會直接回報完成、比較少卡住。還沒做：worker 建立檔案後，決策的步驟看得到檔案的大綱；要「實作」的函式已經存在時，不再呼叫內容通道	Qwen 有 11～16% 的呼叫在空轉	空轉次數、Qwen 的 DevBench 分數
5	**既有專案的回歸檢查**：修改前後各跑一次被改到的模組原本的測試，通過的變少就交回修正或還原；重現問題的測試要先確認在修正前會失敗	未做	4 題「修好了卻弄壞原本的測試」；自己寫的測試照自己的錯誤理解寫	弄壞原本測試的題數、解決數
6	**計畫的檢查器＋交回規劃者改一次**（self-refine 的建議做法）	未做；依你的指示，等測試結束再做	第六節	計畫需要修補的比例、DevBench 介面不符的比例
7	待辦清單的 4 項：介面清單、test-first、網路隔離、amr_base 的真實任務	已記錄，未開始	`doc/待辦清單.md`	見待辦清單
8	**清掉 agent 留下的檔案**：報告列出計畫以外新建的檔案（除錯、重現腳本），由使用者決定要不要刪	未做	v4.2 有 3/23 題的 patch 帶著這種檔案	patch 裡計畫以外的新檔案數

九、後續評測計畫

1. **v4.2 已經量過一次**（E10，4.4 節）：機制有效，分數沒有可信的變化。下一次量測請在第 2、5、6 項做完之後，每個條件跑 3 次。
2. **重複**：SWE-bench 子集每個條件跑 2～3 次。這次每個條件只有 1 次，兩邊解決的題目幾乎不重疊。
3. **擴大題數**：Verified 加到 50 題（接近 Verified mini 的規模）；Multilingual 的 C／C++ 擴大到 15～20 題。
4. **換一組沒看過的題目**：v4.2 是看過這 24 題的結果才改的，要用新的題目確認，避免過度擬合。
5. **amr_base 的真實任務**（待辦 4）：日常使用最有代表性的資料，決定能不能真的交給它。
6. **再試別的模型之前**，先做路線圖第 4 項（讓協議適用於不同模型）。否則量到的是「協議合不合」，而不是模型的能力。

附錄 A：Web Console 的模型與 thinking 設定

在畫面上設定：按標題列的「🧠 模型」，三個角色各有一個模型選單（列出 Ollama 已安裝的模型）和一個 thinking 開關。

角色	用在哪裡	設定存在
主對話（非 coding）	一般對話、維運操作	`logs/model_settings.json`（下次啟動會載入，比環境變數 `WEB_CONSOLE_MODEL` 優先）
Coding 規劃	coding job 的規劃	`logs/coding_settings.json`（Coding 面板的 ⚙️ 也可以設）
Coding worker	coding job 的每一項	同上

- thinking 開關只在模型支援 thinking 時出現；不支援的模型一律送「關」。
- 主對話開了 thinking 時，模型的思考顯示在右欄的「🧠 模型的思考」，不會進入對話。
- 摘要模型沒有另外指定時，會跟著主對話的模型一起換。

指令：

指令	作用			
`/model`	查看三個角色的模型與 thinking 開關			
`/model <模型>`	換主對話的模型，例如 `/model gemma4:26b`			
`/code_model planner\	worker <模型>`	換 Coding 規劃或 worker 的模型		
`/think main\	planner\	worker on\	off`	切換某個角色的 thinking

環境變數（預設值，畫面上的設定優先）：`AGENT_MAIN_THINK`、`AGENT_CODING_THINK`（規劃）、`AGENT_CODING_WORKER_THINK`，都是 `1` 開、預設關。

在 Isaac Sim 執行時用 32B 模型：

- 預設的 Ollama 設定下，32B 模型放不進 GPU，有一部分會放到 CPU，只剩約 18 tokens/秒。
- 啟動 Ollama 時加上下面兩個設定就放得進去（約 42 tokens/秒）：

```
OLLAMA_FLASH_ATTENTION=1 OLLAMA_KV_CACHE_TYPE=q8_0 ollama serve
```

- gemma4 系列不需要這個設定。

附錄 B：名詞解釋

名詞	白話說明
Harness（框架）	包在語言模型外面的程式：給模型可以用的工具（看檔案、改檔案、跑測試…）、決定流程（先規劃、再一項一項做）、檢查模型的成果、出錯時要它重做。模型負責「想」，harness 負責「把關與執行」。
Orchestrator／worker	本 harness 的分工：orchestrator（規劃者）把任務拆成幾項；每一項交給一個 worker（一個獨立的模型對話）去改程式。
既有專案／從零寫	既有專案：在已經存在的程式裡照需求或問題修改（英文常叫 brownfield）。從零寫：只有規格，從空的資料夾寫出整個專案（greenfield）。
SWE-bench	2024 年發表、最常用的「既有專案」基準：每一題是 GitHub 上真實的 issue（問題回報）與修正它的程式碼；agent 看得到 issue 與修正前的整個專案，要交出修改。**Verified** 是其中 500 題經過人工確認的 Python 子集；**Multilingual** 是 9 種語言的 300 題（含 C、C++）。
Issue／patch	Issue：使用者回報的問題或需求。Patch：一份「改了哪些檔案、哪幾行」的修改（`git diff` 的輸出）。
解決（resolved）	SWE-bench 的計分：套上 agent 的 patch 後，原本失敗、修好後應該通過的測試（FAIL_TO_PASS）全部通過，**而且**原本通過的測試（PASS_TO_PASS）沒有被弄壞。
Docker 映像／容器	映像：一個已經裝好系統、套件、專案與測試環境的「快照」。容器：用映像啟動的隔離環境。每一題用官方的映像，確保每個人的環境一樣。
定位	找到該改的檔案與位置。大型專案有幾千個檔案，定位本身就是難題。
BM25（關鍵字檢索）	搜尋引擎常用的關鍵字相關度算法。「不用 harness」的對照組用它，從 issue 的文字找出最相關的原始檔放進提示。
SEARCH/REPLACE	修改大檔案的方式：模型寫出「要找的原文」與「換成的新內容」，由程式找到原文後替換。原文寫得跟檔案不一樣時就「對不上」。
空轉的修改	模型送回的新內容跟檔案目前的內容一模一樣，等於沒有修改、白花一次呼叫。
DevBench	2024 年發表的長任務基準；我們用它的「實作」任務：只給需求文件、UML 圖與架構設計，要寫出整個 repo，再用官方的測試計分。
單元測試／驗收測試	單元測試檢查個別函式、類別的行為（很依賴介面名稱有沒有照規格）；驗收測試從使用者的角度跑整個程式。
thinking 模式／推理模型	模型先輸出一段「思考過程」再回答。推理模型（例如 DeepSeek-R1）是專門用強化學習訓練出長鏈思考的模型。思考會花時間與 token，不一定讓答案更好。
蒸餾（distillation）	用大模型的輸出訓練小模型，讓小模型模仿大模型的行為。這次用的 deepseek-r1:32b 是把 R1（671B）的推理蒸餾到 Qwen2.5-32B。

名詞	白話說明
量化（Q4_K_M）	把模型的參數從 16-bit 壓成約 4-bit，記憶體需求變成約四分之一，準確度會下降一點。Ollama 預設就是這種。
KV cache	模型處理長文字時暫存的中間結果，context 越長佔的顯示記憶體越多；可以量化成 8-bit（q8_0）省一半。
MoE／dense	MoE（混合專家）模型每個 token 只啟用一部分參數，速度快（gemma4:26b 每次約用 4B）；dense 模型每個 token 都用全部的參數（Qwen、R1 的 32B）。
Self-refine／自我修正	讓模型「產生 → 自己檢查 → 修改」。研究發現：有外部回饋（測試、程式檢查）時有效；只靠模型自己看，常常改不對。
LLM-Modulo	一種規劃的設計：模型產生計畫，由外部的程式或驗證器檢查，把具體的錯誤交回模型修改。
災難性回溯	某些寫法的正規表示式在對不上時，會嘗試指數多種組合，幾千行的檔案就可以跑好幾分鐘甚至卡死。
95% 信賴區間（CI）	「真正的數值大概落在這個範圍」。樣本少時範圍很寬；兩個條件的區間大部分重疊，就代表數據還不足以說誰比較好。
配對比較／McNemar 檢定／符號檢定	兩個條件做同樣的題目，只看「一個對、一個錯」的題目：一邊贏得夠多才算有差別（p 值越小越不像運氣，通常 $p < 0.05$ 才算有把握）。
對照組（baseline）	用來比較的參考條件。本研究的「不用 harness」對照組：同一個模型、同樣的資訊，直接請它寫出修改。harness 比對照組多做到的部分，就是 harness 的貢獻。
過度擬合（overfitting）	一直看著同一組題目改系統，分數會變好，但換一組沒看過的題目不一定一樣好。
資料污染	評測的題目或答案出現在模型的訓練資料裡，分數會偏高。公開的 GitHub 題目都有這個風險。
Token／num_ctx	模型處理文字的單位／一次能看到的最大長度（本研究 32,768 個 token）。